

1. Creating a SparkSession

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("InterviewCheatsheet") \
    .getOrCreate()
```

Explanation:

A SparkSession is the entry point for working with Spark. It manages the cluster connection and creates DataFrames.

2. Creating a DataFrame

```
data = [("Shivani", 25), ("Amit", 30), ("Raj", 28)]
columns = ["Name", "Age"]

df = spark.createDataFrame(data, columns)
df.show()
```

Explanation:

A DataFrame is similar to a table in SQL or a Pandas DataFrame.

3. Reading a CSV File

```
df = spark.read.csv("data.csv", header=True,  
inferSchema=True)  
df.show()
```

Explanation:

header=True considers the first row as a header, and inferSchema=True automatically detects data types.

4. Writing a CSV File

```
df.write.csv("output.csv", header=True,  
mode="overwrite")
```

Explanation:

Writes DataFrame to a CSV file with a header.
"overwrite" replaces existing data.

5. Schema Definition

```
from pyspark.sql.types import StructType,  
StructField, StringType, IntegerType
```

```
schema = StructType([  
    StructField("Name", StringType(), True),  
    StructField("Age", IntegerType(), True)  
])
```

```
df = spark.createDataFrame(data,  
schema=schema)  
df.printSchema()
```

Explanation:

Defines a structured schema for better control over data types.

6. Filtering Data

```
df.filter(df.Age > 26).show()
```

 **Explanation:**

Filters rows where Age > 26.

7. Selecting Columns

```
df.select("Name").show()
```

 **Explanation:**

Returns only the Name column.

8. Adding a New Column

```
from pyspark.sql.functions import lit

df = df.withColumn("Country", lit("India"))
df.show()
```

Explanation:

lit("India") adds a new column with a constant value.

9. Renaming a Column

```
df = df.withColumnRenamed("Age", "Years")
df.show()
```

Explanation:

Renames column Age to Years.

10. Dropping a Column

```
df = df.drop("Country")  
df.show()
```

Explanation:

Drops the Country column.

11. Handling Null Values

```
df = df.na.fill({"Age": 0})  
df.show()
```

Explanation:

Fills NULL values in the Age column with 0.

12. GroupBy and Aggregation

```
df.groupby("Name").agg({"Age": "max"}).show()
```

Explanation:

Finds the maximum age for each name.

13. Sorting Data

```
df.orderBy(df.Age.desc()).show()
```

Explanation:

Sorts data in descending order of Age.

14. Joining Two DataFrames

```
data2 = [("Shivani", "Pune"), ("Amit", "Mumbai")]  
df2 = spark.createDataFrame(data2, ["Name",  
"City"])
```

```
df.join(df2, "Name").show()
```

 **Explanation:**

Performs an inner join on the Name column.

15. Using UDF (User Defined Function)

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def greet(name):
    return "Hello, " + name

greet_udf = udf(greet, StringType())

df = df.withColumn("Greeting",
greet_udf(df.Name))
df.show()
```

Explanation:

Defines a custom function and applies it to a DataFrame column.

16. Exploding Arrays

```
data = [("Shivani", ["Python", "Spark"]), ("Amit",  
["Scala", "Java"])]  
df = spark.createDataFrame(data, ["Name",  
"Skills"])
```

```
from pyspark.sql.functions import explode  
df = df.withColumn("Skill", explode(df.Skills))  
df.show()
```

Explanation:

Converts array values into separate rows.

17. Window Functions

```
from pyspark.sql.window import Window
from pyspark.sql.functions import rank

window_spec =
Window.partitionBy("City").orderBy(df.Age.desc(
))
df.withColumn("Rank",
rank().over(window_spec)).show()
```

 **Explanation:**

Ranks records within each city.

18. Pivoting Data

```
df.groupBy("Name").pivot("City").sum("Age").
show()
```

 **Explanation:**

Converts row values into column headers.

19. Handling Duplicates

```
df.dropDuplicates(["Name"]).show()
```

 **Explanation:**

Removes duplicate names.

20. Cache & Persist

```
df.cache()  
df.show()
```

 **Explanation:**

Stores the DataFrame in memory for faster processing.

21. Repartitioning

```
df = df.repartition(2)  
df.show()
```

Explanation:

Redistributes data into 2 partitions.

22. Writing Data in Parquet Format

```
df.write.parquet("output.parquet")
```

Explanation:

Parquet format is optimized for big data processing.

23. Broadcast Joins

```
from pyspark.sql.functions import broadcast

df = df.join(broadcast(df2), "Name")
df.show()
```

Explanation:

Optimizes joins when one DataFrame is small.

24. Handling Date & Timestamp

```
from pyspark.sql.functions import
current_date, current_timestamp

df = df.withColumn("Date",
current_date()).withColumn("Timestamp",
current_timestamp())
df.show()
```

Explanation:

Adds current date and timestamp.

25. Converting DataFrame to Pandas

```
pdf = df.toPandas()  
print(pdf)
```

Explanation:

Converts a Spark DataFrame to a Pandas DataFrame.

26. Reading JSON Data

```
df = spark.read.json("data.json")  
df.show()
```

Explanation:

Reads JSON data.

27. Using Explode with JSON

```
df = df.withColumn("data",  
explode(df.json_column))  
df.show()
```

Explanation:

Expands JSON data.

28. Writing Data in JSON Format

```
df.write.json("output.json")
```

Explanation:

Saves the DataFrame as a JSON file.